Propositional Logic

Logic is a formalism in which to express knowledge and ways of reasoning. More precisely, a logic is a formal language with well-defined syntax and semantics that allows us to represent the world via logical statements (i.e., formulae) and reason systematically about the implications of those statements.

The key idea underlying logic is that the *form* of an argument is distinct from its subject matter. That is, many arguments, be them about numbers, grid worlds, or current events, share the same underlying structure. Logic provides a means of characterizing that structure.

Logical reasoning begins with a series of assertions—assumptions or **premises**—about the world, and allows us to ask the question: what do these premises entail? That is, what else must be true if these premises are true. Logic is thus a means of discovering the consequences of our knowledge.

Logic will be of use to us in at least two distinct ways. The first is as a language in which to represent (search and/or optimization) *problems*, and the second is a language in which to represent *knowledge*. A knowledge-based AI agent maintains a **knowledge base**, meaning a collection of logical statements (i.e., assertions) about the world, and then applies logical reasoning to infer the implications of its knowledge.

This lecture concerns the syntax and semantics of propositional logic. In upcoming lectures, we will use logic both to represent search and optimization problems, and to build knowledge-based AI agents.

1 Syntax

The syntax of a language is the rules that govern construction of its sentences, given an alphabet. In a logical language, these sentences are called **statements** or **formulae**[1] (formulas).

The alphabet of propositional logic contains the constants $\{\top, \bot\}$, a set of propositional variables $\{P, Q, R, \ldots\}$, and a set of connectives $\{\neg, \vee, \wedge, \rightarrow, \leftarrow, \leftrightarrow\}$.

The following inductive definition describes the formulas of propositional logic. Complex formulas involve one or more of the connectives; atomic formulas do not.

- the constants $\top$, $\bot$ are **atomic** formulas
- the propositional variables $P, Q, R, \ldots$ are **atomic** formulas
- if A and B are formulas, then **complex** formulas are formed as follows:
  - $\neg(A)$ is a formula
  - $(A \vee B)$ is a formula
  - $(A \wedge B)$ is a formula
  - $(A \rightarrow B)$ is a formula
  - $(A \leftarrow B)$ is a formula
  - $(A \leftrightarrow B)$ is a formula

[1] Latin was the original *lingua franca* (universal language) of early European philosophers and logicians, e.g., Leibniz. This Latin plural is still widely used today, even now that English has been adopted as our *lingua franca*.

In the absence of parentheses, the precedence of the connectives is as follows: $\neg$, $\wedge$, $\vee$, $\rightarrow$, $\leftarrow$, and $\leftrightarrow$.

Propositional variables represent propositions and connectives represent ways of combining propositions. A **proposition** is an assertion with a definitive truth value: it is either true or false. For example, "$1+1=2$" is a true assertion in our world, while "$1+1=3$" is not. Moreover, if P represents the proposition "the lights are on" and Q represents the proposition "the door is closed," then $P \wedge Q$ represents the formula "the lights are on *and* the door is closed," while $\neg P$ represents the formula "the lights are *not* on."

2 Semantics

The semantics of a language give meaning to its formulas. The formulas of propositional logic are assigned one of two values, or "meanings," either true (T) or false (F).

Atomic formulas are assigned meaning by an **interpretation** $\mathcal{I} = \langle D, M \rangle$, where

- $D = \{\text{T}, \text{F}\}$ is the **domain** of values, or "meanings"
- M is an **assignment** that maps atomic formulas into domain D *s.t.* $M(\top) = \text{T}$ and $M(\bot) = \text{F}$

A logical formula A is evaluated by a semantic **evaluation function** v, which takes as input an interpretation $\mathcal{I}$ and outputs an element of domain D. We abbreviate $v(\mathcal{I}, A) = \text{T}$ by $\mathcal{I} \models A$ and $v(\mathcal{I}, A) = \text{F}$ by $\mathcal{I} \not\models A$. The former is read "$\mathcal{I}$ satisfies A" or "$\mathcal{I}$ is a **model** of A;" the latter is read inversely.

Given interpretation $\mathcal{I}$ and formula A, $\mathcal{I} \models A$ is defined inductively as follows:

- $\mathcal{I} \models \top$ and $\mathcal{I} \not\models \bot$, since $M(\top) = \text{T}$ and $M(\bot) = \text{F}$
- $\mathcal{I} \models P$ iff $M(P) = \text{T}$
- $\mathcal{I} \models \neg A$ iff $\mathcal{I} \not\models A$
- $\mathcal{I} \models A \vee B$ iff $\mathcal{I} \models A$ or $\mathcal{I} \models B$
- $\mathcal{I} \models A \wedge B$ iff $\mathcal{I} \models A$ and $\mathcal{I} \models B$
- $\mathcal{I} \models A \rightarrow B$ iff if $\mathcal{I} \models A$ then $\mathcal{I} \models B$
- $\mathcal{I} \models A \leftarrow B$ iff $\mathcal{I} \models A$ whenever $\mathcal{I} \models B$
- $\mathcal{I} \models A \leftrightarrow B$ iff if $\mathcal{I} \models A$ then $\mathcal{I} \models B$ and if $\mathcal{I} \models B$ then $\mathcal{I} \models A$

This definition of semantic evaluation function can be more succinctly expressed via truth tables. The first few columns describe the various assignments. There are two rows in the truth table on the left, capturing the two possible assignments of a single propositional variable A; and there are four rows in the truth table on the right, capturing the four possible assignments of two propositional variables A and B. The remaining columns, which pertain to complex rather than atomic formulas, describe the inductive cases.

A	$\neg A$
T	F
F	T

A	B	$A \vee B$	$A \wedge B$	$A \rightarrow B$	$A \leftarrow B$	$A \leftrightarrow B$
T	T	T	T	T	T	T
T	F	T	F	F	T	F
F	T	T	F	T	F	F
F	F	F	F	T	T	T

N.B. Logical implication is not equivalent in meaning to causal implication as it is used in natural language. Let A represent the formula "pigs can fly," and let B represent the formula "Al Gore is the president of the United States."[2] Now consider the formula $A \rightarrow B$. In natural language, one would interpret this formula to mean that the existence of flying pigs (not the voice of the people) is one condition sufficient for Al Gore to be president of the United States. In propositional logic, however, $A \rightarrow B$ is vacuously true.

Definition

- A is *satisfiable* iff it has a model: i.e., there exists $\mathcal{I}$ *s.t.* $\mathcal{I} \models A$
- A is *valid* iff all interpretations are models: i.e., for all $\mathcal{I}$, $\mathcal{I} \models A$.
- A is *unsatisfiable* iff no interpretation is a model: i.e., for all $\mathcal{I}$, $\mathcal{I} \not\models A$

The question of whether a formula of propositional logic has a model is the famous **satisfiability** problem, and the subject of the next lecture.

The notation $\models$ is polymorphic. Thus far, we've used it to define a relationship between an interpretation and a formula. But it can also be used to define a relationship between two formulas: e.g., $A \models B$. The meaning of this latter use is, "every model of A is also a model of B."

The following exercises make use of the fact that $\top$ is satisfied by all interpretations, while $\bot$ is satisfied by none. Consequently, $\mathcal{I} \models \top$ always holds, while $\mathcal{I} \models \bot$, equivalently, $\mathcal{I} \not\models \top$, never does.

Exercise Show that A is satisfiable iff $A \not\models \bot$.

Solution

$$
\begin{array}{ll}
 & A \text{ is satisfiable} \\
\text{iff} & \text{there exists } \mathcal{I} \textit{ s.t. } \mathcal{I} \models A \\
\text{iff} & \text{there exists } \mathcal{I} \textit{ s.t. } \mathcal{I} \models A \text{ and } \mathcal{I} \models \top \\
\text{iff} & \text{it is not the case that for all } \mathcal{I},\ \mathcal{I} \not\models A \text{ or } \mathcal{I} \models \bot \\
\text{iff} & \text{it is not the case that for all } \mathcal{I},\ \text{if } \mathcal{I} \models A, \text{then } \mathcal{I} \models \bot \\
\text{iff} & A \not\models \bot
\end{array}
$$

Corollary A is unsatisfiable iff $A \models \bot$.

Exercise Show that A is valid iff $\top \models A$.

Solution

$$
\begin{array}{ll}
 & A \text{ is valid} \\
\text{iff} & \text{for all } \mathcal{I},\ \mathcal{I} \models A \\
\text{iff} & \text{for all } \mathcal{I},\ \mathcal{I} \not\models \top \text{ or } \mathcal{I} \models A \\
\text{iff} & \text{for all } \mathcal{I},\ \text{if } \mathcal{I} \models \top, \text{then } \mathcal{I} \models A \\
\text{iff} & \top \models A
\end{array}
$$

Valid formulas are also called **tautologies**, or logical truths, since they are true under all interpretations. Unsatisfiable formulas are also called **contradictions**, or logical falsities: e.g., $A \wedge \neg A$. There is no interpretation $\mathcal{I}$ *s.t.* $\mathcal{I} \models A \wedge \neg A$.

Some examples of tautologies are listed below:

[2] These notes were compiled in the very early 2000s, when Professor Greenwald started her career at Brown.

- Law of Double Negation
  $\neg\neg A \leftrightarrow A$
- DeMorgan's Laws
  $\neg(A \vee B) \leftrightarrow (\neg A \wedge \neg B)$
  $\neg(A \wedge B) \leftrightarrow (\neg A \vee \neg B)$
- Distributive Laws
  $(A \wedge B) \vee C \leftrightarrow (A \vee C) \wedge (B \vee C)$
  $(A \vee B) \wedge C \leftrightarrow (A \wedge C) \vee (B \wedge C)$
- Associative Laws
  $(A \vee B) \vee C \leftrightarrow A \vee (B \vee C)$
  $(A \wedge B) \wedge C \leftrightarrow A \wedge (B \wedge C)$
- Commutative Laws
  $A \vee B \leftrightarrow B \vee A$
  $A \wedge B \leftrightarrow B \wedge A$
- Material Implication
  $A \rightarrow B \leftrightarrow \neg A \vee B$
  $\neg A \rightarrow B \leftrightarrow A \vee B$

Exercise Using truth tables, establish that these laws are in fact tautologies.